🌲 DAY 5: ARROW WORKFLOWS & TOPIC TREES

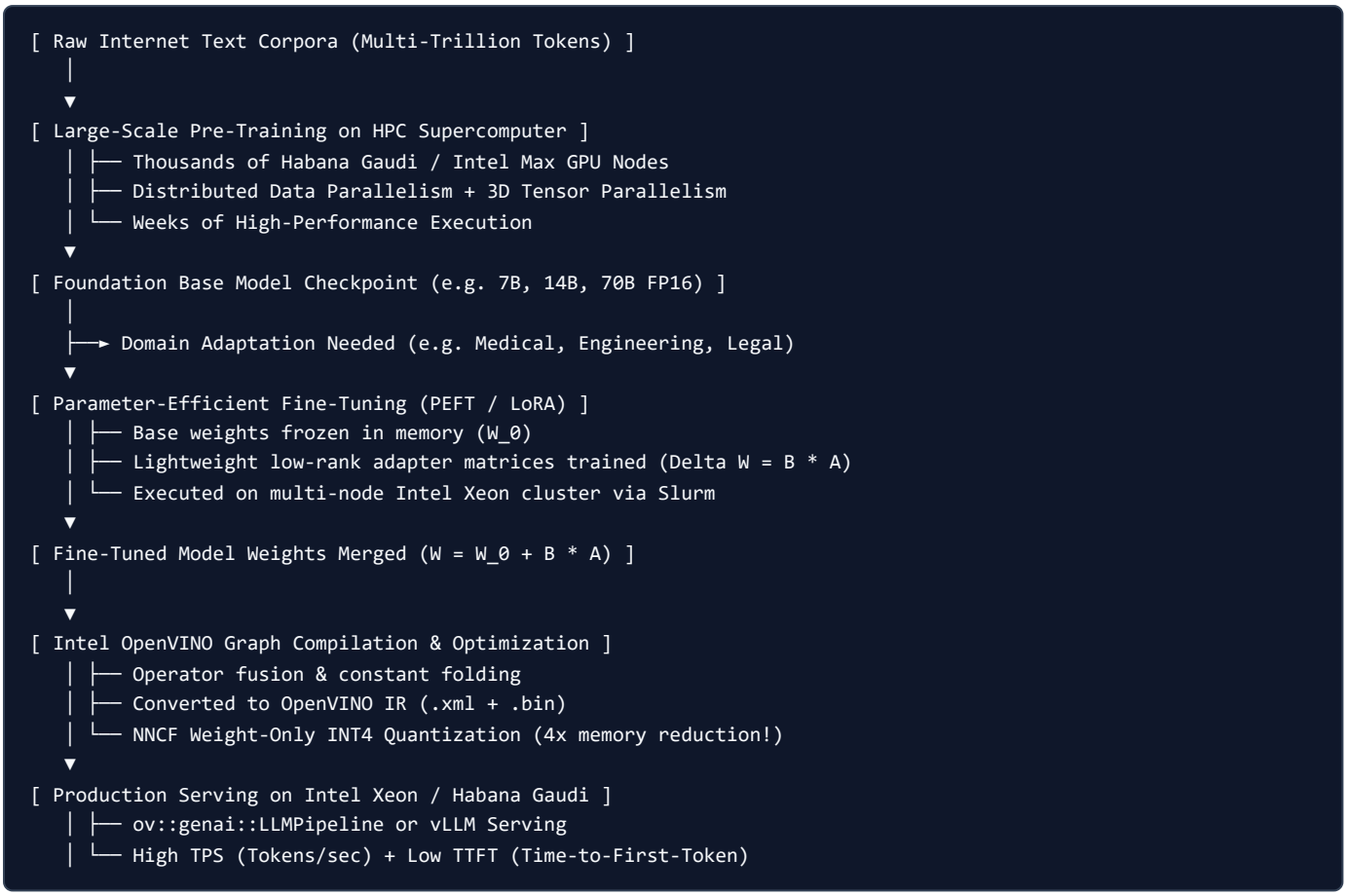
Generative AI, Large Language Models (LLMs), Habana Gaudi & OpenVINO Acceleration

Visual Step-by-Step Execution Pathways for HPC Engineers & Trainers

📌 TABLE OF TOPIC TREES

1. Tree 1: End-to-End LLM Supercomputing Lifecycle
2. Tree 2: Auto-Regressive Decoding: Prefill vs Decode Memory Wall Flow
3. Tree 3: Intel Xeon AMX (Advanced Matrix Extensions) Systolic Execution Flow
4. Tree 4: Habana Gaudi 2 / Gaudi 3 Silicon Architecture & SynapseAI Pipeline
5. Tree 5: Intel OpenVINO Model Ingestion & NNCF INT4 Quantization Pipeline
6. Tree 6: LoRA (Low-Rank Adaptation) Matrix Decomposition & Backprop Flow
7. Tree 7: Multi-Node Slurm Distributed Fine-Tuning & Microservice Serving

TREE 1: End-to-End LLM Supercomputing Lifecycle



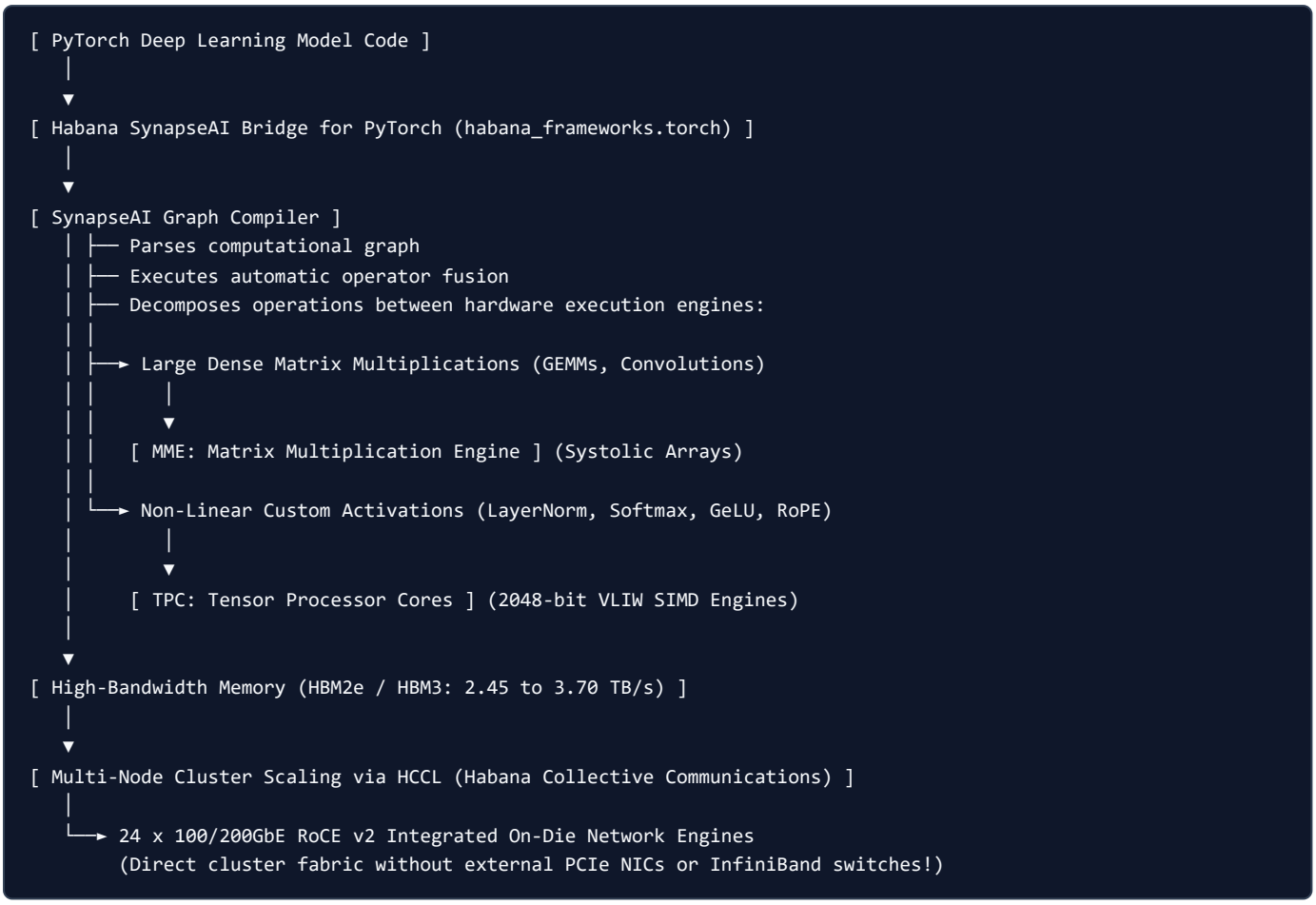
TREE 2: Auto-Regressive Decoding: Prefill vs Decode Memory Wall Flow

```
[ User Submits Prompt: "Explain High Performance Computing" (L = 16 tokens) ]
   |
   ▼
+-------------------------------------------------------------------------+
| PHASE 1: PREFILL PHASE (TIME TO FIRST TOKEN - TTFT)                     |
+-------------------------------------------------------------------------+
   |
   ├── Highly COMPUTE-BOUND (Arithmetic Intensity > 15 FLOP/byte)
   ├── Processes all 16 prompt tokens simultaneously in a single forward pass
   ├── Large matrix-matrix multiplications (GEMM) saturate Intel AMX / AVX-512
   |
   ├──► Computes Key & Value projections for all 16 tokens
   ├──► Allocates and writes KV-Cache in DRAM
   |
   ▼
[ First Response Token Emitted: "High" (TTFT Measured in Milliseconds) ]
   |
   ▼
+-------------------------------------------------------------------------+
| PHASE 2: DECODE PHASE (TOKENS PER SECOND - TPS)                         |
+-------------------------------------------------------------------------+
   |
   ├── Strictly MEMORY-BANDWIDTH BOUND (Arithmetic Intensity ≈ 0.5 FLOP/byte)
   |
   ├──► Loop Iteration 1:
   |    ├── Input: Token "High" (1 token)
   |    ├── Entire 14 GB Model Weights read from DRAM into Cache!
   |    ├── Appends new Key/Value to existing KV-Cache in RAM
   |    └── Emits Token 2: "Performance"
   |
   ├──► Loop Iteration 2:
   |    ├── Input: Token "Performance" (1 token)
   |    ├── Entire 14 GB Model Weights read from DRAM into Cache AGAIN!
   |    ├── Appends new Key/Value to KV-Cache
   |    └── Emits Token 3: "Computing"
   |
   ├──► Loop Iteration N:
   |    └── Continues until <|im_end|> token emitted
   |
   ▼
[ Complete Generated Response Streamed to User ]
[ Performance Dictated by Memory Bandwidth (GB/s), NOT Compute TFLOPs! ]
```

TREE 3: Intel Xeon AMX Systolic Execution Flow

```
[ Deep Learning Forward Pass encounters GEMM (C = C + A * B) ]
   |
   ▼
[ Linux Kernel AMX Permission Request via arch_prctl ]
   |
   ▼
[ Configure Tile Architecture via ldtilecfg instruction ]
   | ├── Sets dimensions of 2D Tile Registers TMM0 - TMM7
   | └── Configures rows and column bytes (up to 16 rows x 64 bytes)
   |
   ▼
[ Load Matrices into Tile Registers ]
   | ├── tileloadd TMM0, [Matrix A pointer in L1/L2 Cache]
   | └── tileloadd TMM1, [Matrix B pointer in L1/L2 Cache]
   |
   ▼
[ Execute Systolic Matrix Multiplication via TMUL Unit ]
   | ├── tdpbf16ps TMM2, TMM0, TMM1 (BF16 inputs ──► FP32 accumulators)
   | ├── Or: tdpbusd TMM2, TMM0, TMM1 (INT8 inputs ──► INT32 accumulators)
   | |
   | └── TMUL executes 1,024 INT8 ops or 512 BF16 ops in ONE clock cycle!
   |
   ▼
[ Store Result Tile back to Memory / Cache ]
   | └── tilestored [Matrix C destination], TMM2
   |
   ▼
[ Execution proceeds to next layer with 8x higher throughput than AVX-512! ]
```

TREE 4: Habana Gaudi Silicon Architecture & SynapseAI Pipeline



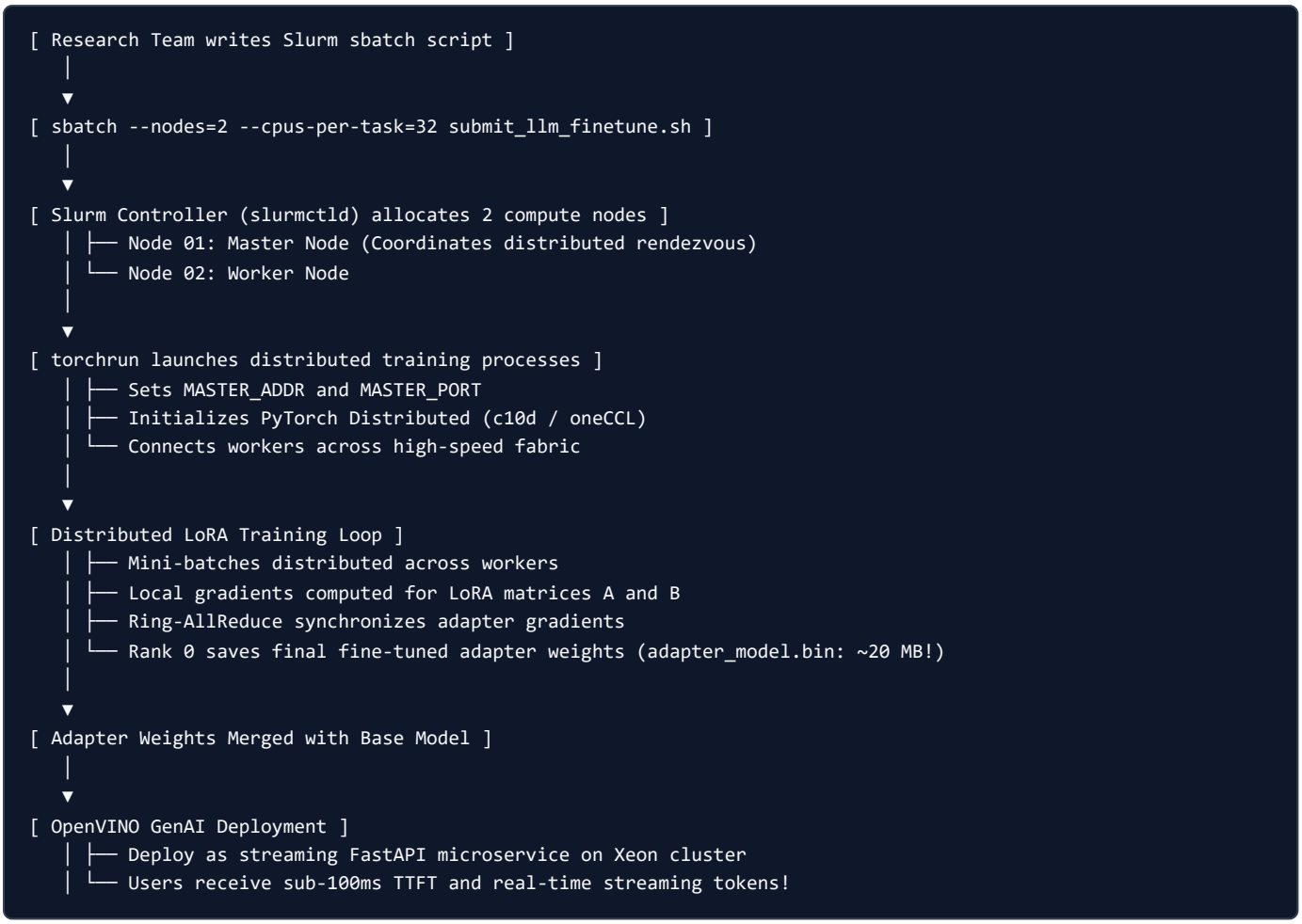
TREE 5: Intel OpenVINO Model Ingestion & NNCF INT4 Quantization Pipeline

```
[ Pre-Trained HuggingFace Transformer Model (FP16 / BF16: ~14 GB) ]
   |
   ▼
[ OpenVINO Model Conversion: ov::convert_model(model, example_inputs) ]
   |
   ▼
[ Graph Optimization Pass ]
   | ├── Constant folding (precomputing static shapes)
   | ├── Dead code elimination
   | └── Operator fusion (Linear + Bias + ReLU merged)
   |
   ▼
[ Base OpenVINO Intermediate Representation (IR) Saved ]
   | ├── model.xml (XML Graph Topology Blueprint)
   | └── model.bin (Contiguous Binary Weights: ~14 GB)
   |
   ▼
[ NNCF Weight-Only Compression: nncf.compress_weights() ]
   | ├── Evaluates weight sensitivity across layers
   | ├── Applies Asymmetric INT4 Quantization (group_size = 128)
   | └── Compresses 16-bit floats to 4-bit integer indices + scales
   |
   ▼
[ Quantized OpenVINO IR Saved ]
   | ├── model_int4.xml (Updated Graph Topology with Dequantize Ops)
   | └── model_int4.bin (Compressed Binary Weights: ~3.5 GB! [4x Reduction])
   |
   ▼
[ High-Performance Inference via ov::genai::LLMPipeline ]
   | ├── 4x Less Data transferred from DRAM per generated token
   | └── 2.5x to 4x Higher Tokens Per Second (TPS) on Intel Xeon!
```

TREE 6: LoRA (Low-Rank Adaptation) Matrix Decomposition & Backprop Flow

```
[ Pre-Trained Linear Layer Weight Matrix: W_0 (Dimension: d x k) ]
   |
   ▼
[ Freeze Base Weights: W_0.requires_grad = False ]
   | (Zero gradients computed, zero optimizer memory allocated for W_0!)
   |
   ▼
[ Introduce Low-Rank Bottleneck Adapter Matrices: A and B ]
   | ├── Matrix A: Dimension r x k (Initialized with Gaussian random values)
   | ├── Matrix B: Dimension d x r (Initialized to all zeros)
   | └── Rank r << min(d, k) (e.g. d=4096, k=4096, r=8)
   |
   ▼
[ Forward Pass for Input Vector x (Dimension d) ]
   |
   ├── Branch 1: h_base = x * W_0 (Frozen base projection)
   |
   └── Branch 2: h_adapter = (alpha / r) * (x * B * A) (Low-rank adaptation)
   |
   ▼
[ Combine Outputs: h_final = h_base + h_adapter ]
   |
   ▼
[ Loss Computation & Backward Pass ]
   |
   ├── Gradients propagate ONLY into Matrix A and Matrix B!
   ├── Total Trainable Parameters reduced by 99.6%!
   └── Optimizer VRAM drops from Terabytes to Megabytes!
```

TREE 7: Multi-Node Slurm Distributed Fine-Tuning & Microservice Serving



GLA University HPC Masterclass | Day 5 Arrow Workflows & Topic Trees Completed